Implementación comparativa de la estructura de una base de datos

Red Social Estudiantil Pascualina

Integrantes:
Mariana Díaz Lastre
José David Celis Cepeda

Grupo:
7

Asignatura:
Bases de Datos I (G100)

Docente:
Oralia Cortes Grajales

Institución Universitaria Pascual Bravo
2026

1. Introducción

Este informe documenta la adaptación física de la base de datos de la Red Social Estudiantil Pascualina en PostgreSQL, MySQL y Microsoft SQL Server. Se toma como punto de partida el modelo de la Tarea 3, que define diez tablas para estudiantes, perfiles, publicaciones, grupos, eventos, intereses y sus relaciones. La implementación conserva las entidades y reglas principales, y adapta la sintaxis y los tipos de datos a cada motor.

La versión entregada es una propuesta de implementación y documentación. Las capturas y los resultados deben completarse con ejecuciones reales de los scripts en cada SGBD; no se presentan como verificados los motores que el equipo aún no haya ejecutado.

2. Selección de los SGBD y justificación

2.1 PostgreSQL

Se conserva PostgreSQL porque el equipo ya cuenta con una propuesta inicial y porque es un motor de código abierto con restricciones CHECK, claves foráneas, transacciones y soporte avanzado para datos semiestructurados mediante JSONB. El índice GIN permite acelerar determinadas búsquedas sobre documentos JSONB. pgAdmin 4 facilita la ejecución y exploración de la estructura.

2.2 MySQL

Se propone MySQL 8.0.16 o posterior por su uso extendido en aplicaciones web, su integración con herramientas como MySQL Workbench y el tipo JSON nativo. La identidad se expresa con AUTO_INCREMENT y las tablas usan InnoDB para mantener integridad referencial. Se debe confirmar la versión instalada, especialmente para validar restricciones CHECK.

2.3 Microsoft SQL Server

Se propone SQL Server 2016 o posterior por su integración con el ecosistema Microsoft y la herramienta SQL Server Management Studio (SSMS). En las versiones consideradas, el contenido JSON se almacena como NVARCHAR(MAX) y se valida con ISJSON. La identidad se declara con IDENTITY(1,1).

SGBD	Ventajas para el proyecto	Consideraciones
PostgreSQL	Código abierto; JSONB; índices GIN; restricciones robustas.	Requiere conocer operadores y funciones específicas de PostgreSQL.
MySQL	Tipo JSON; InnoDB; adopción amplia en aplicaciones web.	AUTO_INCREMENT y funciones JSON tienen sintaxis propia; verificar versión.
SQL Server	SSMS; integración con herramientas Microsoft; ISJSON.	JSON se representa como texto validado en las versiones consideradas; sintaxis propia.

3. Continuidad con el modelo de la Tarea 3

Se mantienen diez tablas: estudiante, perfil, publicacion, grupo, evento, interes, estudiante_grupo, estudiante_evento, estudiante_interes y seguidor. El diseño contempla trece claves foráneas, la relación uno a uno entre estudiante y perfil, relaciones muchos a muchos mediante tablas asociativas y una restricción que impide que un estudiante se siga a sí mismo.

Para alinear los archivos, se adoptaron los tamaños y reglas documentados en el PDF de la Tarea 3: estudiante.nombre y apellido VARCHAR(50), correo VARCHAR(100), programa VARCHAR(80), semestre SMALLINT; perfil.biografia VARCHAR(300), foto VARCHAR(150), contacto VARCHAR(100); publicacion.contenido VARCHAR(500); grupo.nombre_grupo VARCHAR(80) sin unicidad obligatoria; descripciones de grupo/evento VARCHAR(300); interes.nombre_interes VARCHAR(50) único y descripción VARCHAR(200). Esta reconciliación debe ser aprobada por el equipo si existen cambios posteriores acordados por el docente.

Se corrigió la referencia de estudiante_evento.id_evento para apuntar a evento.id_evento. En el diccionario, los nombres de columnas y las relaciones deben coincidir exactamente con los scripts respectivos.

4. Del modelo lógico al físico

Elemento	Decisión física
Identificadores	INT autogenerado: GENERATED BY DEFAULT AS IDENTITY (PostgreSQL), AUTO_INCREMENT (MySQL) e IDENTITY(1,1) (SQL Server).
Claves primarias compuestas	Se usan en las tablas asociativas y en seguidor para impedir relaciones duplicadas.
Integridad referencial	Las trece claves foráneas enlazan las tablas conforme al modelo. CASCADE se usa cuando la fila dependiente pierde sentido; RESTRICT/NO ACTION protege grupos y eventos creados por estudiantes.
Reglas de dominio	CHECK limita semestre a 1–12 y evita auto-seguimiento.
Fechas	TIMESTAMP en PostgreSQL; TIMESTAMP/DATETIME en MySQL; DATETIME2 en SQL Server, según la necesidad.
Datos semiestructurados	JSONB en PostgreSQL; JSON en MySQL; NVARCHAR(MAX) con ISJSON en SQL Server.

4.1 Índices y rendimiento

Las claves primarias y restricciones UNIQUE crean los índices necesarios según cada motor. PostgreSQL incluye un índice GIN sobre perfil.datos_extra para determinadas consultas JSONB. No se debe asumir que este índice es portable a MySQL o SQL Server: los índices sobre propiedades JSON deben definirse en función de las consultas y de las capacidades de la versión instalada. Antes de añadir índices a todas las claves foráneas, conviene revisar los planes de consulta y las necesidades reales de la aplicación.

5. Datos semiestructurados: JSON, Big Data e IoT

El atributo perfil.datos_extra permite registrar información opcional que puede variar entre estudiantes, como habilidades, deportes, idiomas o ubicación aproximada. Un ejemplo es un documento con una lista de habilidades y un objeto de ubicación que incluya latitud y longitud. Al ser atributos variables, no siempre conviene crear una columna relacional nueva por cada característica.

En un escenario relacionado con IoT, una aplicación universitaria podría recibir desde dispositivos móviles datos contextuales o coordenadas asociadas a actividades del campus. El documento JSON facilita conservar campos variables y consultarlos posteriormente. No obstante, JSON por sí solo no convierte la solución en una arquitectura Big Data ni sustituye un sistema de ingestión, controles de

privacidad, validación de coordenadas, políticas de retención o tipos geoespaciales especializados cuando estos sean necesarios.

Las implementaciones tienen diferencias: PostgreSQL usa JSONB, MySQL ofrece JSON nativo y SQL Server almacena el documento como texto Unicode validado con ISJSON en las versiones consideradas. El índice GIN es específico de PostgreSQL. Para comparar rendimiento, el equipo debe ejecutar consultas equivalentes sobre datos de prueba comparables.

6. Implementación paso a paso

1. Crear una base de datos vacía en el motor correspondiente y confirmar la versión del SGBD.
2. Abrir la herramienta de administración: pgAdmin 4 para PostgreSQL, MySQL Workbench o cliente equivalente para MySQL, y SSMS para SQL Server.
3. Abrir y ejecutar el script DDL correspondiente desde la carpeta ddl. No ejecutar un script de un motor en otro motor.
4. Consultar el explorador de objetos y verificar que aparezcan las diez tablas.
5. Consultar las restricciones para comprobar las claves primarias, las trece claves foráneas, UNIQUE y CHECK.
6. Ejecutar las pruebas de datos válidos y, por separado, las pruebas negativas que deben ser rechazadas.
7. Guardar capturas reales de cada paso e indicar versión, herramienta, fecha y resultado observado.
8. Actualizar los diccionarios físicos si una corrección realizada durante la ejecución modifica el script.

Figura 1. Ejecución del DDL en PostgreSQL

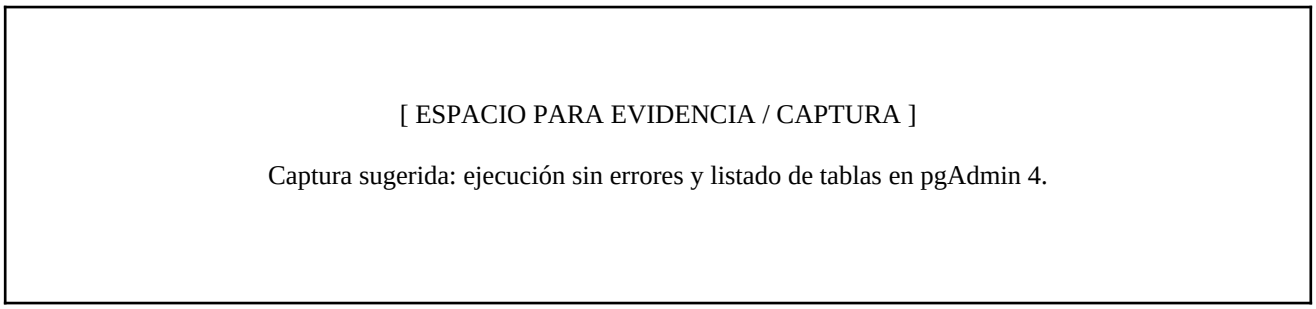


Figura 2. Ejecución del DDL en MySQL

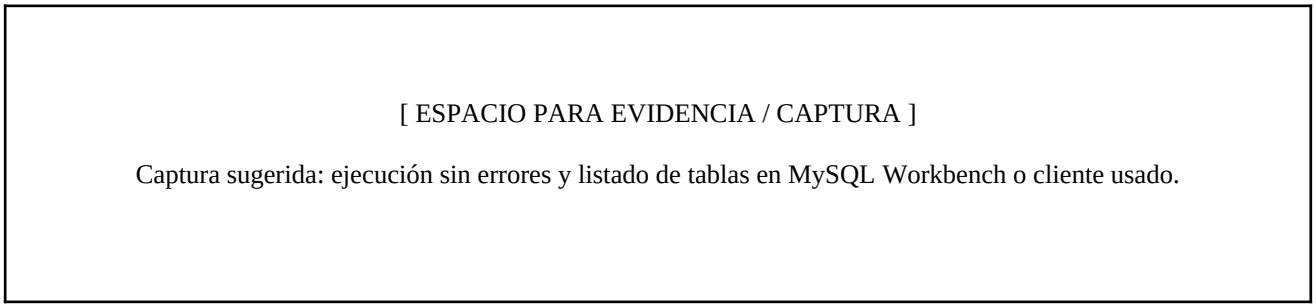


Figura 3. Ejecución del DDL en SQL Server

[ESPACIO PARA EVIDENCIA / CAPTURA]

Captura sugerida: ejecución sin errores y listado de tablas en SSMS.

7. Pruebas y evidencias

El archivo pruebas_postgresql.sql incluye datos de ejemplo, consultas de comprobación y una transacción que revierte los registros de prueba. Debe ejecutarse en una base de datos de pruebas vacía; se recomienda revisar los identificadores usados en las inserciones antes de ejecutarlo en una base que ya contenga registros. Las pruebas negativas (correo duplicado, semestre fuera de rango y auto-seguimiento) se deben ejecutar individualmente y registrar el mensaje real del motor.

Para MySQL y SQL Server, el equipo debe realizar pruebas equivalentes: inserción válida, duplicación de correo, semestre inválido, referencia inexistente, perfil duplicado y auto-seguimiento. Anotar el resultado observado; no afirmar que una prueba pasó hasta verla ejecutarse.

Figura 4. Resultado de consultas y datos de prueba

[ESPACIO PARA EVIDENCIA / CAPTURA]

Incluir una captura con consultas de verificación y resultados visibles.

Figura 5. Restricciones rechazando datos inválidos

[ESPACIO PARA EVIDENCIA / CAPTURA]

Incluir una captura del mensaje de error real para al menos una restricción CHECK y una UNIQUE por motor probado.

8. Dificultades, ventajas y desventajas

Dificultades técnicas previstas para validar durante la práctica: adaptar la sintaxis de identidad, equivalencias entre tipos de fecha y representación JSON, acciones referenciales y diferencias de herramientas. Esta lista no debe presentarse como experiencia observada hasta que los integrantes ejecuten y documenten los scripts.

SGBD	Ventajas esperadas	Limitaciones o dificultades que

		deben comprobarse
PostgreSQL	JSONB, GIN, restricciones y herramientas gratuitas.	Sintaxis específica de JSONB y administración de roles/esquemas.
MySQL	JSON nativo, InnoDB y amplia documentación.	Verificar versión y comportamiento de CHECK; índices JSON según consulta.
SQL Server	SSMS y herramientas de administración integradas.	Representación JSON como texto validado en las versiones elegidas; restricciones de cascada y licenciamiento según edición.

Después de ejecutar los motores, cada integrante debe reemplazar las dificultades previstas por las que realmente encontró, incluyendo el mensaje de error o captura que lo respalde.

9. Conclusiones

9.1 Conclusión grupal — propuesta para revisar

El proceso evidencia que un modelo lógico común puede implementarse en varios SGBD, pero cada motor exige adaptar la sintaxis, los tipos de datos y algunas reglas de integridad. Mantener una fuente común para las entidades y documentar las diferencias físicas ayuda a reducir inconsistencias entre scripts y diccionarios. El uso de JSON permite representar atributos variables del perfil, aunque debe acompañarse de validación, criterios de privacidad e índices diseñados para consultas reales.

9.2 Conclusiones individuales — completar con experiencia real

Mariana Díaz Lastre: [Completar en primera persona: qué parte implementó, qué dificultad real encontró, qué aprendió y qué mejoraría.]

José David Celis Cepeda: [Completar en primera persona: qué parte implementó, qué dificultad real encontró, qué aprendió y qué mejoraría.]

Otros integrantes: [Agregar un párrafo por cada integrante, si corresponde.]

9.3 Propuestas de mejora

- Automatizar un conjunto común de datos de prueba para los tres motores.
- Comparar tiempos de consulta con un volumen de datos controlado y la misma consulta lógica.
- Crear índices adicionales solo cuando las consultas y los planes de ejecución lo justifiquen.
- Documentar las versiones exactas de los SGBD, herramientas, resultados y errores observados.
- Definir medidas de seguridad y privacidad para correo, contacto y ubicación.
- Evaluar tipos geoespaciales especializados si las coordenadas se convierten en una funcionalidad central.

10. Organización del repositorio Git

El repositorio debe contener la carpeta tarea4 con diccionarios, ddl, informe y video. Se incluyen el diccionario genérico y tres diccionarios físicos separados, además de un Excel consolidado para facilitar revisión. El informe final debe guardarse como PDF. En la carpeta video debe incluirse el enlace de YouTube y el guion de sustentación. La estructura final debe revisarse con la guía del curso antes de publicar.

Ruta	**Contenido**
tarea4/diccionarios/	Diccionario genérico y tres diccionarios físicos; también

	Excel consolidado.
tarea4/ddl/	postgresql.sql, mysql.sql, sql_server.sql y pruebas_postgresql.sql.
tarea4/informe/	Informe técnico en PDF y versión editable DOCX.
tarea4/evidencias/	Capturas reales de ejecución, estructura y pruebas por SGBD.
tarea4/video/	Guion y archivo con enlace de YouTube por completar.

Referencias

Elmasri, R. y Navathe, S. (2007). Fundamentos de sistemas de bases de datos (5.ª ed., pp. 317–344). Pearson Education.

Fadhel, S. A. y Jameel, E. A. (2022). A comparison between NoSQL and RDBMS: storage and retrieval. Minar International Journal of Applied Sciences and Technology, 4(3), 172–184. https://doi.org/10.47832/2717-8234.12.18

Hasan, M., Panidi, E. y Badenko, V. (2019). Comparative evaluation of NoSQL and relational databases performance while analyzing semi-structured geospatial data. GEOBALCANICA. https://doi.org/10.18509/gbp.2019.64

Herrera-Ramírez, J., Treviño-Villalobos, M. y Víquez-Acuña, L. (2021). Un motor de almacenamiento híbrido para datos geoespaciales utilizando paradigmas NoSQL y SQL. Revista Tecnología en Marcha, 34(1), 40–54. https://doi.org/10.18845/tm.v34i1.4822

Mannino, M. V. (2007). Administración de bases de datos: diseño y desarrollo de aplicaciones (3.ª ed.). McGraw-Hill Interamericana.

Microsoft Learn. (2025, 12 de junio). ¿Qué es SQL Server Management Studio (SSMS)? https://learn.microsoft.com/es-es/ssms/sql-server-management-studio-ssms

Treviño-Villalobos, M., Víquez-Acuña, L., Quirós-Oviedo, R. y Víquez-Acuña, O. (2022). Evaluación del tiempo de respuesta de un geoservicio utilizando una base de datos híbrida y distribuida. Revista Colombiana de Computación, 23(1), 34–43. https://doi.org/10.29375/25392115.4228